

Phase 5 Practice Questions

Easy Level (Q1 - Q4)

Question 1 — Synchronous vs Asynchronous

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 2000);

console.log("End");
```

Predict Output

Expected Output

```
Start
End
Timer
```

Scenario

A loading message appears while data is being fetched.

What is this question asking?

Understand that `setTimeout()` does not block JavaScript.

JavaScript schedules the timer and continues executing the remaining code.

Concepts Tested

- Synchronous vs Asynchronous
 - setTimeout
-

Question 2 — Simple Callback

Create a function:

```
greet (name)
```

and another function:

```
welcome(callback)
```

that calls the callback with:

```
"Ritik"
```

Expected Output

```
Hello Ritik
```

Scenario

A website welcomes users after login.

What is this question asking?

Pass a function into another function and execute it later.

Concepts Tested

- Callbacks
- Functions as Arguments

Question 3 — setTimeout with Arguments

Create:

```
function greet(name) {  
  console.log(`Hello ${name}`);  
}
```

Use `setTimeout()` to print:

```
Hello Ritik
```

after 2 seconds.

Concepts Tested

- setTimeout
- Passing Arguments

Question 4 — Stop a Timer

Create a timer:

```
setTimeout(...)
```

scheduled for 5 seconds.

Cancel it before execution.

Expected Output

```
Nothing should print.
```

Scenario

User closes a popup before it appears.

Concepts Tested

- `clearTimeout()`
-

Moderate Level (Q5 - Q8)

Question 5 — Countdown Timer

Create a countdown:

```
5
4
3
2
1
Done
```

using:

```
setInterval()
```

Scenario

OTP resend countdown.

What is this question asking?

Use repeated execution and stop it automatically when complete.

Concepts Tested

- `setInterval`

- clearInterval

Question 6 — Fake API Call

Create:

```
fetchUser(callback)
```

After 2 seconds return:

```
{  
  id:1,  
  name:"Ritik"  
}
```

using:

```
setTimeout()
```

Expected Output

```
Fetching User...  
  
{id:1,name:"Ritik" }
```

Scenario

Simulate server response.

Concepts Tested

- Callbacks
- Async Operations

Question 7 — Create Your First Promise

Create a Promise that:

After 2 seconds:

```
resolve("Data Received")
```

Expected Output

```
Data Received
```

Scenario

API request completed successfully.

What is this question asking?

Learn Promise creation and resolution.

Concepts Tested

- Promise
- resolve()

Question 8 — Promise Rejection

Create a Promise that rejects with:

```
"Server Down"
```

Handle the error properly.

Expected Output

Server Down

Concepts Tested

- `reject()`
- `catch()`

Hard Level (Q9 - Q12)

Question 9 — Promise Chaining

Create:

```
addTen ( num )
```

which returns a Promise.

Starting from:

```
0
```

perform:

```
+10  
+10  
+10
```

using Promise chaining.

Expected Output

```
10  
20
```

Scenario

Processing data through multiple stages.

What is this question asking?

Understand how `.then()` chains work.

Concepts Tested

- Promise Chaining
- `then()`

Question 10 — Async/Await Conversion

Convert this:

```
function fetchData() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Data Received");  
    }, 2000);  
  });  
}
```

```
fetchData()  
  .then(data => {  
    console.log(data);  
  })  
  .catch(err => {  
    console.log(err);  
  });
```


into:

```
async/await
```

Expected Output

Same behavior.

Scenario

Modern React applications use async/await.

Concepts Tested

- async
- await
- try/catch

Question 11 — Event Loop Prediction

Predict the output:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve()
  .then(() => {
    console.log("C");
  });

console.log("D");
```

Expected Output

A
D
C
B

What is this question asking?

Understand:

Sync Code
↓
Microtasks
↓
Macrotasks

Concepts Tested

- Event Loop
- Promise Queue
- Callback Queue

Question 12 — Fetch + JSON (Mini Project)

Fetch user data from:

<https://jsonplaceholder.typicode.com/users/1>

Requirements:

1. Use `fetch()`
2. Use `async/await`

3. Check `response.ok`
 4. Convert response using `response.json()`
 5. Print user's name
-

Expected Output

Leanne Graham

Scenario

Profile page loading user information.

What is this question asking?

This combines everything:

1. Async/Await
 2. Fetch API
 3. JSON Parsing
 4. Error Handling
-

Concepts Tested

- `fetch()`
 - `async/await`
 - `response.json()`
 - `response.ok`
 - `try/catch`
-

Bonus Interview Question

Predict the output:

```

console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve()
  .then(() => {
    console.log("Promise 1");
  })
  .then(() => {
    console.log("Promise 2");
  });

console.log("End");

```

Expected Output

```

Start
End
Promise1
Promise2
Timeout

```

Why?

1. Sync Code
 - Start
 - End
2. Microtasks
 - Promise 1
 - Promise 2

3. Macrotasks

Timeout

This is one of the most commonly asked Event Loop interview questions. 🚀